TradeFlow — Build Notes

Personal reference for the TradeFlow MVP build. Where things stand, how it is wired, and what comes next.

Generated 2026-08-27 • Repo: github.com/ChamithMendis/trade_flow • Current state: **Phase 0 complete & pushed** (commit `d0b06d8`)

1. What TradeFlow is

A real-time trading **simulator** — a portfolio project meant to show full-stack engineering beyond CRUD: asynchronous order lifecycle, real-time updates, data consistency, idempotency, background jobs. It does **not** touch a real exchange or real money.

Core flow: user places an order → API validates & persists it as `NEW` → a background worker (Exchange Simulator) partially fills / fills / rejects it → every state change is persisted and pushed to the browser over WebSockets → the portfolio updates.

Order lifecycle: `NEW → PROCESSING → PARTIALLY_FILLED… → FILLED`, with terminal branches `REJECTED` and `CANCELLED`.

2. Tech stack

Layer	Choice	Notes
Monorepo	npm workspaces	One tool fewer than pnpm; `apps/*` + `packages/*`
Web	React 19 + TypeScript + Vite	React Router, TanStack Query (server state), Zustand (client state)
Web UI	Tailwind CSS + shadcn/ui, Recharts	Added in later phases
API	NestJS 11 + TypeScript	Modular monolith — not microservices
Database	PostgreSQL 16 + Prisma	`Decimal` for money, never float
Async jobs	BullMQ + Redis 7	Exchange simulator worker
Real time	Socket.IO	Per-user rooms, JWT-authenticated handshake
Auth	JWT + Argon2	Access token only for MVP
Validation	class-validator + class-transformer	DTO validation, global pipe
Tests	Jest + Supertest	Nest default
Infra (local)	Docker Compose	Postgres + Redis containers
Shared	`@tradeflow/shared-types`	Enums + WS event names, compiled to CommonJS

3. Repository structure

```
tradeflow/
├── package.json            # npm workspaces + root scripts
├── tsconfig.base.json      # shared strict TS compiler options
├── .prettierrc.json .editorconfig .gitattributes(LF) .gitignore .nvmrc
├── .env.example            # DB / Redis / JWT / Vite vars
├── docker-compose.yml      # postgres:16-alpine + redis:7-alpine
├── docs/
│   ├── architecture.md     # module map + phase status
│   └── decisions.md        # architecture decision log (ADRs)
├── packages/
│   └── shared-types/       # @tradeflow/shared-types
│       └── src/index.ts    # enums + WsEvent constants → dist/ (CJS)
└── apps/
    ├── api/                # @tradeflow/api  (NestJS)
    │   └── src/            # main.ts, app.module.ts, app.controller.ts (/health)
    └── web/                # @tradeflow/web  (Vite React-TS, starter page for now)
```

4. Full phase plan

Each phase is independently runnable and gets its own commit. Recommended order: 0 → 8 in sequence.

#	Phase	Key tech / what happens	Status
0	Foundation	Monorepo, TS, lint/format, Docker infra, `/health`	Done
1	Database schema	Prisma schema + migrations: User, Instrument, Order, Execution, Portfolio, Position, OrderEvent. `executionReference` unique = idempotency backbone. Seed script.	Next
2	Auth	NestJS: Argon2 hashing, JWT, Passport guard, global `ValidationPipe`, throttler. `/auth/register`, `/auth/login`, `/auth/me`. React: React Router, TanStack Query, Zustand auth store, protected routes.	Planned
3	Market + live prices	Instruments API, `@Interval()` price random-walk, Socket.IO broadcast of `market.price.updated`. React market dashboard with live rows + sparklines.	Planned
4	Order entry	BUY/SELL form, MARKET/LIMIT, validation (cash, holdings, qty, instrument), persist as `NEW`, order list + detail screens.	Planned
5	Exchange simulator	BullMQ queue + Nest worker. Random outcome: reject / full fill / 2–4 partials. Unique `executionReference` per fill. Persist executions + order events. State machine + idempotency.	Planned
6	Real-time order updates	JWT-authenticated Socket.IO, per-user rooms, emit order/portfolio events. React patches TanStack Query cache from socket events — no polling. Toasts.	Planned
7	Portfolio	Settlement in one transaction with row lock. BUY updates avg cost, SELL reduces position. Market value + unrealized P/L computed on read. Portfolio dashboard (Recharts).	Planned
8	Quality & deploy	Jest tests for the §14 critical scenarios, Swagger at `/docs`, Dockerfiles, GitHub Actions CI, deploy (Railway/Fly + Vercel), README screenshots + demo video.	Planned

Critical test scenarios (Phase 8 acceptance): can't buy without cash; can't oversell; multi-partial-fill path; filled order can't be cancelled; cancelled order ignores later executions; **duplicate execution event doesn't double-update the portfolio**; user A can't read user B's orders/portfolio; sockets deliver only the owner's events.

5. Phase 0 — what was actually built

Root tooling

File	Purpose
`package.json`	Workspaces (`packages/*` , `apps/*`); scripts: `dev` , `build` , `lint` , `format` , `typecheck` , `infra:up/down`
`tsconfig.base.json`	Shared strict compiler options; each workspace extends it
`.prettierrc.json` + `.prettierignore`	Single formatter for the whole repo (single quotes, semicolons, width 100)
`.editorconfig` , `.gitattributes`	LF line endings enforced cross-platform
`.nvmrc`	Node 20
`.env.example`	Placeholders for `DATABASE_URL` , `REDIS_URL` , `API_PORT` , `JWT_SECRET` , `VITE_API_URL` , `VITE_WS_URL`
`docker-compose.yml`	Postgres 16 + Redis 7 (alpine), healthchecks, named volumes
`docs/architecture.md` , `docs/decisions.md`	Architecture map + decision log

packages/shared-types

Exports (from `src/index.ts`), built to CommonJS so both Nest (CJS) and Vite (bundler) resolve it:

- `UserRole` — TRADER, ADMIN
- `InstrumentStatus` — ACTIVE, DISABLED
- `OrderSide` — BUY, SELL
- `OrderType` — MARKET, LIMIT
- `OrderStatus` — NEW, PROCESSING, PARTIALLY_FILLED, FILLED, REJECTED, CANCELLED
- `OrderEventType` — CREATED, PROCESSING, PARTIALLY_FILLED, FILLED, REJECTED, CANCELLED
- `TERMINAL_ORDER_STATUSES` — array of the 3 terminal states
- `WsEvent` / `WsEventName` — the 8 Socket.IO event-name strings from the spec

apps/api (NestJS)

- Renamed package to `@tradeflow/api`
- `main.ts` : reads `API_PORT` (default 3000) and `WEB_ORIGIN` from env; enables CORS
- `app.controller.ts` : added `GET /health` → `{ status: 'ok', uptime }`
- `app.controller.spec.ts` : added a health smoke test (2 tests total, both pass)

apps/web (Vite React-TS)

- Renamed package to `@tradeflow/web` ; added `@tradeflow/shared-types` dependency

- Still the default Vite starter page — no TradeFlow UI yet (that starts Phase 2)

Deviations from the original plan

> The current Vite scaffold ships **oxlint** (not ESLint) and TypeScript 6. Rather than fight the scaffold, each app keeps its own linter (`oxlint` for web, `eslint` for api) and the repo root owns a single **Prettier** config. Recorded as ADR-0002.

Decision log (docs/decisions.md)

ADR	Decision
0001	Modular monolith + npm workspaces; `shared-types` compiled to CJS to prevent web/api contract drift
0002	Per-app linters (oxlint / eslint), root-level Prettier only
0003	Local infra via Docker Compose (Postgres 16 + Redis 7); needs Docker Desktop + WSL2 on Windows

6. How to run & verify (current state)

No database needed until Phase 1, so Docker is not required to check Phase 0.

Setup (once)

```
cd d:/chamith_only_this_remain/Projects/trade_flow
npm install
Copy-Item .env.example .env          # PowerShell (bash: cp .env.example .env)
```

Static checks

```
npm run typecheck                    # all 3 workspaces compile
npm run lint                         # api (eslint) + web (oxlint)
npm test -w @tradeflow/api           # 2 tests pass
npm run build                        # shared-types + api + web build
```

Run the apps

```
npm run dev
# starts 3 processes: types (tsc -w), api :3000, web :5173
# Ctrl+C once stops all three
```

Verify

Check	Command / action	Expected
API health	`curl http://localhost:3000/health`	`{"status":"ok","uptime":…}`
API root	`curl http://localhost:3000/`	`Hello World!`
Web	open `http://localhost:5173`	Vite + React starter page
Shared package	import `OrderStatus` from `@tradeflow/shared-types` in `App.tsx`	resolves, hot-reloads, no error

7. Git & GitHub

- Local branch renamed `master` → `main`
- Remote `origin` → `https://github.com/ChamithMendis/trade_flow.git`
- Phase 0 committed as `d0b06d8` ("Phase 0: monorepo foundation") and pushed
- Workflow going forward: one commit per phase, `git push` after each

8. Pending / before Phase 1

> **Install Docker Desktop for Windows** (enable the WSL2 backend). Phase 1 onwards needs a running PostgreSQL. Then `npm run infra:up` starts Postgres + Redis.

9. Next: Phase 1 — Database schema

1. Add Prisma to `apps/api`, `prisma init`
2. Model the 7 tables from spec §10 with enums and constraints:
   - `User.email` unique
   - `Execution.executionReference` unique — the idempotency backbone
   - `Position` unique on `(portfolioId, instrumentId)`
   - Money columns as `Decimal`
3. First migration (`prisma migrate dev`)
4. Seed: 1 admin, 5 instruments (TFLX, NOVA, APEX, VERT, QUANT); portfolio with starting cash created on registration
5. **Done when:** `prisma studio` shows the seeded data

The assistant will propose the exact schema and file list before writing any code.